

# Projekt: Cloud Programming

## Phase 2

Jonas Habel

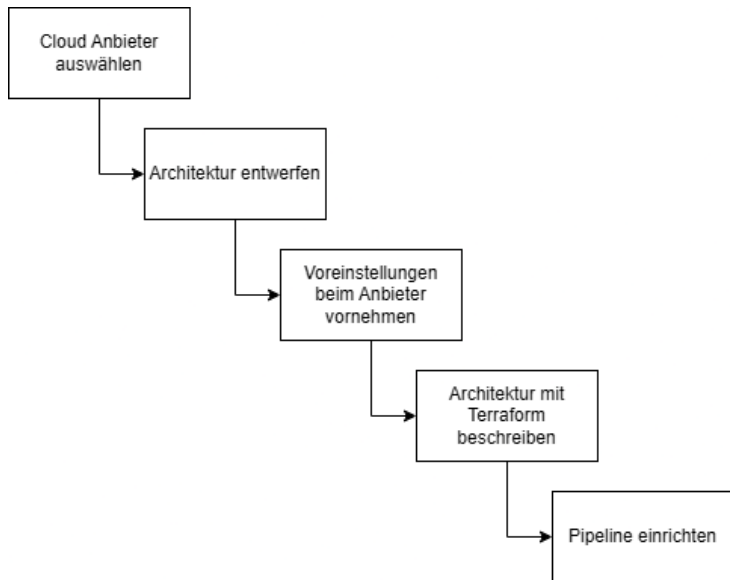
February 3, 2026

# Agenda

- 1 Aufgabenstellung
- 2 Vorgehensweise
- 3 Cloud Anbieter auswählen
- 4 Architekturentwurf
- 5 Voreinstellungen
- 6 Anforderungen und Umsetzung
- 7 Pipeline in GitHub
- 8 Das erfolgreiche Deployment

- Einrichten der Cloud Architektur der Unternehmenswebsite
- Website muss hochverfügbar sein
- Nutzende aus der ganzen Welt sollen keine Verzögerung erfahren
- Automatisches skalieren wenn es viele Anfragen gibt.

# Vorgehensweise

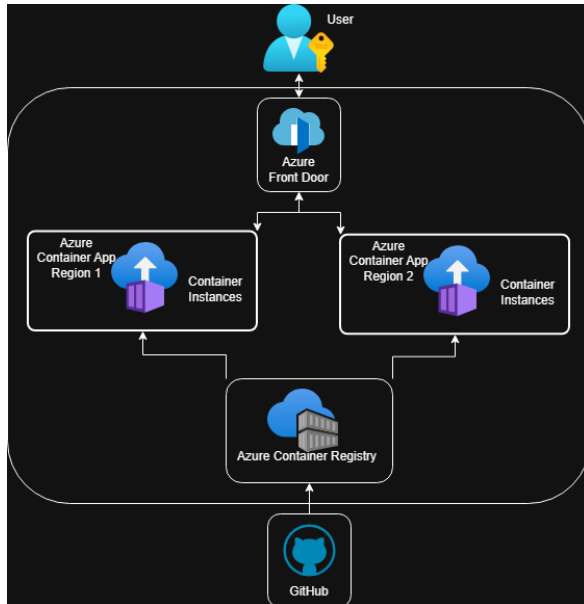


# AWS vs. Azure: Kosten & Anforderungen

| Anforderung               | AWS                                                  | Azure                                                               |
|---------------------------|------------------------------------------------------|---------------------------------------------------------------------|
| Hochverfügbare Website    | Multi-AZ + Load Balancer                             | Container Apps in 2 Regionen + Front Door (Failover/Load-Balancing) |
| Weltweit ohne Verzögerung | CloudFront (CDN) + global DNS                        | Front Door + optional CDN/Caching                                   |
| Auto-Scaling bei Last     | Auto Scaling / ECS/EKS / Lambda                      | Container Apps Autoscaling + Front Door                             |
| Kosten                    | ähnlich; Treiber: Compute + Egress + Multi-AZ/Region | ähnlich; Treiber: Container Apps + Egress + Front Door + 2 Regionen |

## Auswahl

Die Auswahl ist auf Azure gefallen, da die Kosten vergleichbar sind und dieses bereits im Unternehmen Verwendung findet.



- Azure CLI installiert
- Terraform installiert
- Docker installiert
- Azure subscription
- App registration in Azure (Microsoft Entra ID → App registrations)
- Client Secret für die registrierte App für CLI Zugriff
- Federated Credentials der registrierten App hinzufügen für OIDC Auth in der Pipeline

Mit Hilfe des `terroform-local.cmd` Skripts kann der gesamte Bereitstellungsprozess lokal ausgeführt werden. Dazu müssen die folgenden Umgebungsvariables gesetzt werden:

- `ACR_NAME`, `ACR_RESOURCE_GROUP`, `ACR_LOCATION`
- `APP_ID`
- `TENANT_ID`
- `CLIENT_SECRET`
- `IMAGE_NAME`, `IMAGE_TAG`
- `TFSTATE_RESOURCE_GROUP`, `TF_STATE_STORAGE_ACCOUNT`,  
`TF_STATE_CONTAINER`, `TFSTATE_KEY`



# Skript ausführen

Zum Ausführen muss der folgende Befehl vom Root Verzeichnis des Projekts in einer Kommandozeile eingegeben werden:

## Command

```
scripts\terraform-local.cmd
```

Das Skript durchläuft die folgenden Schritte:

- Login in Azure mit Service Principal.
- (optional) Docker-Image bauen und nach ACR pushen.
- Terraform mit Remote-Backend initialisieren.
- Terraform anwenden, um Infrastruktur zu erstellen/aktualisieren.

- Hochverfügbarkeit: Zwei regionale Container Apps (primary/secondary) hinter Front Door

## Azure Front Door

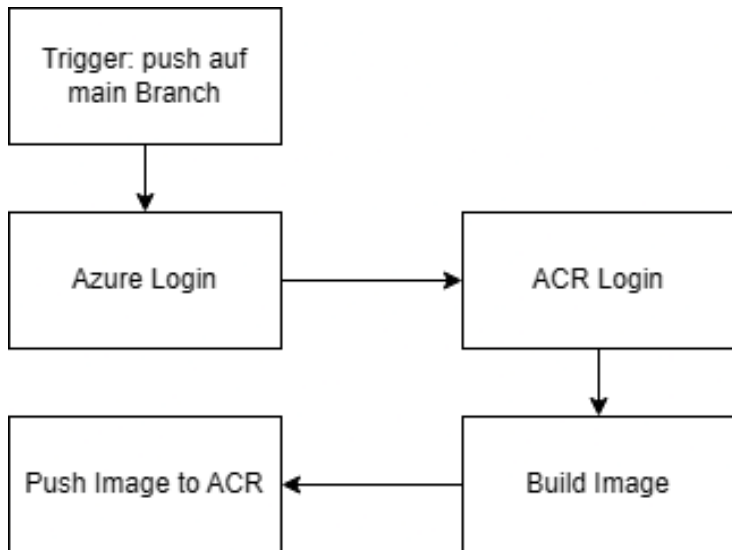
<https://learn.microsoft.com/de-de/azure/frontdoor/front-door-overview>

- Weltweite Besucher: Front Door als globaler Einstiegspunkt mit Geo-Routing und Anycast.
- Autoscaling: Container Apps skalieren automatisch nach HTTP-Last (min/max Replikas)

## Azure Container Apps

<https://learn.microsoft.com/en-us/azure/container-apps/overview>

# Pipeline in GitHub



# Das erfolgreiche Deployment

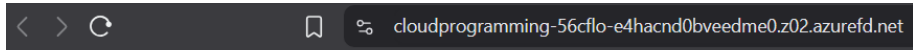
```
azurerm_container_app.primary: Modifying... [id=/subscriptions/.../resourceGroups/cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app1]
azurerm_container_app.secondary: Modifying... [id=/subscriptions/.../resourceGroups/cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app2]
azurerm_container_app.primary: Still modifying... [id=/subscriptions/.../resourceGroups/cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app1, 00m10s elapsed]
azurerm_container_app.secondary: Still modifying... [id=/subscriptions/.../resourceGroups/cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app2, 00m10s elapsed]
azurerm_container_app.secondary: Still modifying... [id=/subscriptions/.../resourceGroups/cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app2, 00m20s elapsed]
azurerm_container_app.primary: Still modifying... [id=/subscriptions/.../resourceGroups/cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app1, 00m20s elapsed]
azurerm_container_app.secondary: Modifications complete after 21s [id=/subscriptions/.../resourceGroups/rg-cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app2]
azurerm_container_app.primary: Modifications complete after 21s [id=/subscriptions/.../resourceGroups/rg-cloudprogramming/providers/Microsoft.App/containerApps/cloudprogramming-app1]
Releasing state lock. This may take a few moments...
```

Apply complete! Resources: 0 added, 2 changed, 0 destroyed.

## Outputs:

```
acr_login_server = "cpregistry2913874.azurecr.io"
acr_name = "cpregistry2913874"
container_app_primary_fqdn = "cloudprogramming-app1.redriver-50d6dae4.westeurope.azurecontainerapps.io"
container_app_secondary_fqdn = "cloudprogramming-app2.yellowdesert-9d06c94e.northeurope.azurecontainerapps.io"
frontdoor_endpoint_host = "cloudprogramming-56cflo-e4hacnd0bveedme0.z02.azurefd.net"
frontdoor_url = "https://cloudprogramming-56cflo-e4hacnd0bveedme0.z02.azurefd.net"
```

```
D:\repos\CloudProgramming>
```



**Hello, World! From the Cloud!!!!**